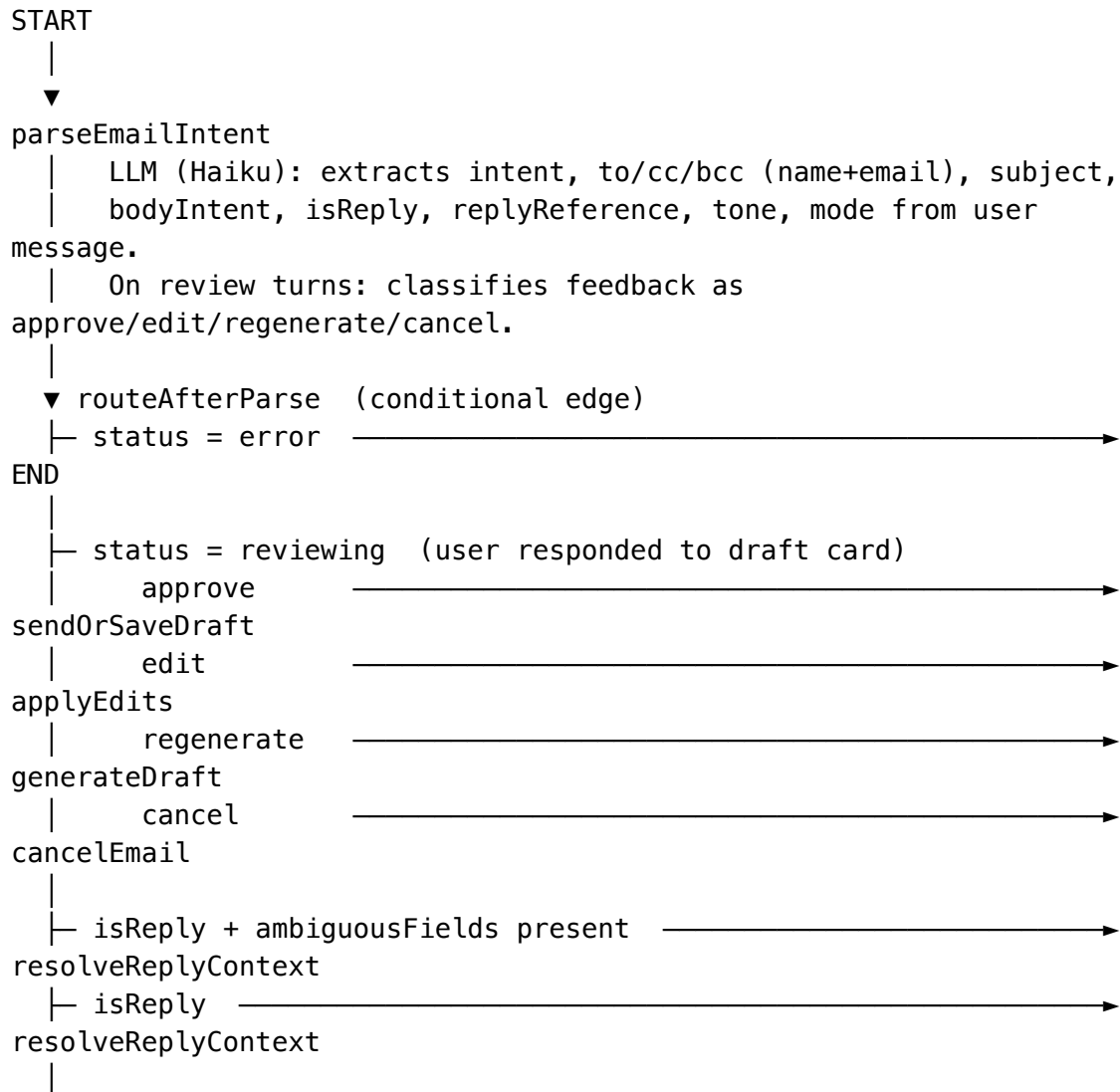


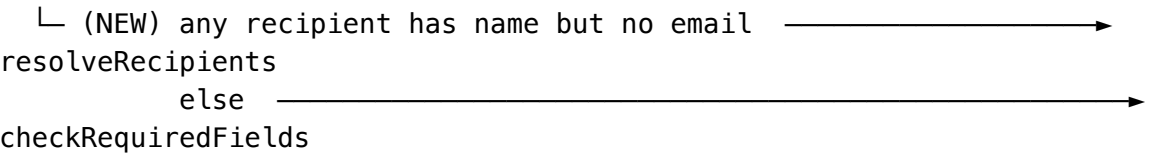
Email Agent RAG Flow

Email Agent — Agentic Flow & RAG Search Implementation

1. Current Agentic Flow

Graph Overview





Node Descriptions

Node	Purpose
parseEmailIntent	LLM extracts all email fields. On review turns, classifies feedback.
resolveReplyContext	RAG (existing) — vector searches inbox for the email the user wants to reply to. Handles confident match / ambiguous / no-match.
resolveRecipients	RAG (new) — looks up email addresses for name-only recipients via SQL metadata scan + vector fallback.
checkRequiredFields	Validates all required fields are present. Routes to clarify or generate.
clarifyMissing	LLM generates a clarifying question for missing/ambiguous fields. Terminal — ends the turn.
generateDraft	LLM (Sonnet) generates the full email draft. Appends to draftHistory.
presentDraftForApproval	Formats and surfaces the draft to the user as an approval card. Terminal — ends the turn.
applyEdits	LLM applies the user’s edit instruction to the current draft.
sendOrSaveDraft	Calls Gmail API to send or save draft. Only node that touches Gmail API.
cancelEmail	Resets state and surfaces cancellation message.

State Fields (key ones)

- emailDetails.to/cc/bcc — array of { email, name } objects
- emailDetails.isReply — boolean
- emailDetails.replyReference — natural language reference (“Raj’s email about the deadline”)
- originalEmail — populated by resolveReplyContext: messageId, threadId, from, subject, bodyPreview
- replyContextCandidates — used for disambiguating reply target OR recipient address
- draftHistory — append-only array of { version, subject, body, source }
- status — idle → parsing → resolving → clarifying → generating → reviewing → editing → sending → sent | saved_draft | cancelled | error

2. RAG Search — Reply Context (Existing)

File: backend/src/service/email/replyContextService.js

Trigger: routeAfterParse routes to resolveReplyContext when emailDetails.isReply === true.

How it works:

```

replyReference ("Raj's email about the deadline")
  |
  ▼
vectorStore.similaritySearch(replyReference, 3, { source: "gmail" })
  |   PGVector cosine similarity against all indexed Gmail messages
  |
  ▼
score comparison
  |─ top score < 0.70                → clarify: "I couldn't find that
email..."
  |─ top[0].score - top[1].score < 0.05 → ambiguous: show 2
candidates
  |─ confident                        → populate state.originalEmail,
continue

```

Threshold constants:

```

CONFIDENCE_THRESHOLD = 0.7
AMBIGUITY_DELTA      = 0.05

```

3. RAG Search — Recipient Email Lookup (New)

Problem

parseEmailIntent correctly returns { email: null, name: "Priya" } when only a name is given (per prompt rules). But sendOrSaveDraft does .map(r => r.email).filter(Boolean) — name-only recipients are silently dropped.

Solution: resolveRecipients node

A new node inserted between routeAfterParse and checkRequiredFields that resolves name-only recipients via two-strategy RAG search.

Implementation

A. Service — recipientResolver.js

Strategy 1 — SQL metadata scan (primary)

```

const sqlResults = await pool.query(
  `SELECT
    metadata->>'from' AS from_addr,
    metadata->'to' AS to_list,
    metadata->'cc' AS cc_list,
    metadata->'bcc' AS bcc_list
  FROM documents
  WHERE source = 'gmail'
  AND (
    metadata->>'from' ILIKE $1
    OR (metadata->'to')::text ILIKE $1
    OR (metadata->'cc')::text ILIKE $1
    OR (metadata->'bcc')::text ILIKE $1
  )
  LIMIT 50`,
  [`%${name}%`]
);

```

Count how many times each email address appears alongside the queried name. Most frequent = highest confidence.

Strategy 2 — Vector similarity fallback

```

vectorStore.similaritySearch(`emails involving ${name}`, 5, { source:
  "gmail" })

```

Used when the name is not in structured metadata fields (e.g., nickname or partial name).

Address parsing handles both RFC 5322 format ("John Doe <john@x.com>") and plain email strings.

B. Node — resolveRecipients.js

```

for each name-only recipient in to/cc/bcc:
  recipientResolver.findEmailByName(name)
  |
  |— no result OR score < 0.75
  |   → stillUnresolved list
  |
  |— 2 candidates, score delta < 0.08
  |   → clarifyMissing: "Which John did you mean? 1.
john@a.com 2. john@b.com"
  |
  |— confident single match
  |   → fill in email field, continue

if stillUnresolved > 0:
  → clarifyMissing: "I couldn't find an email for "Priya". Could you
provide it?"

else:

```

- mergeEmailDetails with resolved addresses
- status = "resolving"
- checkRequiredFields

Threshold constants:

CONFIDENCE_THRESHOLD = 0.75
 AMBIGUITY_DELTA = 0.08

C. Graph wiring

In graph.js:

```
import { resolveRecipients } from "../nodes/resolveRecipients.js";

workflow.addNode("resolveRecipients", resolveRecipients);

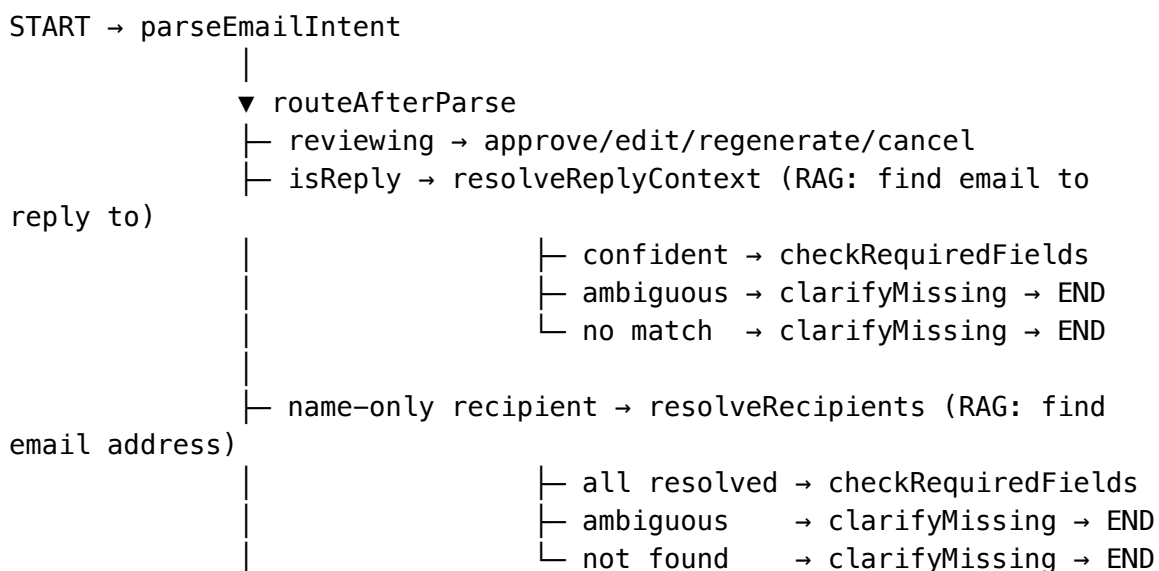
workflow.addConditionalEdges("resolveRecipients", (state) => {
  if (state.status === "error") return END;
  if (state.status === "clarifying") return "clarifyMissing";
  return "checkRequiredFields";
});
```

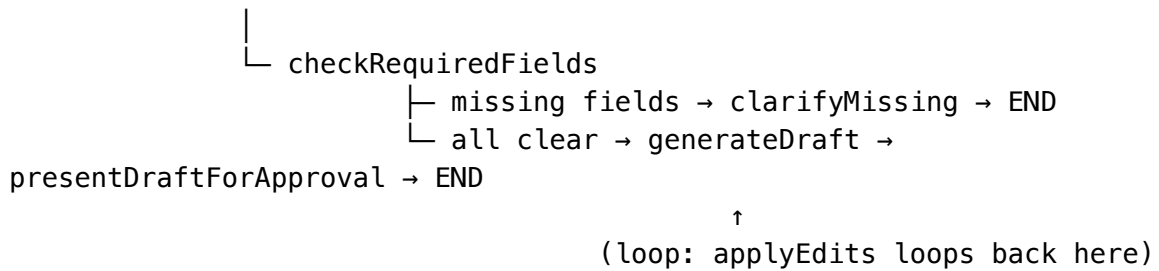
In routeAfterParse.js (add before the final return "checkRequiredFields"):

```
const needsRecipientResolution = ["to", "cc", "bcc"].some((f) =>
  state.emailDetails[f].some((r) => !r.email && r.name)
);
if (needsRecipientResolution) return "resolveRecipients";

return "checkRequiredFields";
```

4. Updated Full Flow (with both RAG hooks)





5. Important Note on Metadata Format

Before implementing, verify the actual to/cc/bcc storage format in your DB:

```

SELECT metadata->'to', metadata->'from'
FROM documents
WHERE source = 'gmail'
LIMIT 3;

```

- If stored as string array ["John Doe <john@x.com>"] → use the parseAddress RFC 5322 parser
- If stored as object array [{"email":"john@x.com","name":"John"}] → read item.email / item.name directly in harvestList

The replyContextService accesses r.metadata.from (flat, not r.metadata.gmail.from), which suggests the sync pipeline stores metadata fields at the top level.